CSED233 · 데이터구조 · 포스텍

Ch9. Binary Search Tree

1. BST(이진 탐색 트리) 정의

핵심 Property

- **Binary Tree**이면서 각 노드가 (key, value) 쌍 저장, **key는 unique**
- **BST property** (핵심):
  - 모든 노드 n에 대해:
    - `n.key` > left subtree 모든 key
    - `n.key` < right subtree 모든 key
  - → 재귀적으로 모든 subtree가 BST여야 함
- 별칭: **ordered/sorted binary tree**

왜 쓰나?

- **Search, Insert, Delete** 모두 평균 $O(\log n)$ (랜덤 삽입 순서의 기대값; skewed 최악 O(n))
- 동시에 **Sorted order output** (inorder 순회)도 $O(n)$
- → Dynamic dictionary의 기본형. Hash table과 달리 **순서 유지**.

2. Search

알고리즘

```
Search(x, T):
    if T == NULL: return NOT_FOUND
    if x == T.key: return T
    if x <  T.key: return Search(x, T.left)
    else:          return Search(x, T.right)
```

- 매 step에서 한쪽 subtree 버림 → 높이만큼만 탐색

시간복잡도

- **Balanced BST**: $O(\log n)$
- **Skewed (linked list처럼 한쪽으로 치우침)**: $O(n)$ ← **worst case**
- → 균형 유지가 핵심 과제 (Ch10 AVL에서 해결)

Q1. 같은 key 집합, 높이 다를 수 있나?

> 힌트

삽입 순서가 BST의 모양(높이)을 결정한다.

> 정답

개념은 힌트 참고.

- **가능**. 삽입 순서에 따라 높이가 완전히 달라짐.
- 예: {1,2,3,4,5}
  - 순서대로 insert → skewed tree 높이 4 ($O(n)$)
  - 3,1,5,2,4 순서 → 균형 있는 tree 높이 2 ($O(\log n)$)

3. Inorder(중위 순회) Traversal(순회) = Sorted Output

핵심 사실

- **BST의 inorder traversal = key를 오름차순으로 반환**

증명 직관

- Inorder 정의: `left → node → right`
- BST property: left의 모든 key < node < right의 모든 key
- → 재귀적으로 inorder는 '작은 것부터 큰 것까지' 순서대로 출력

Q2. Inorder만으로 BST 재구성?

> 힌트

Inorder가 주는 정보는 정렬된 나열뿐이다.

> **정답**

개념은 힌트 참고.

- **불가**. Inorder는 **항상 sorted sequence**만 주므로 원래 구조는 재구성 불가.
- 예: `1,2,3` inorder → tree가 여러 가지 가능
- → 일반 Binary Tree에서는 식별값이 모두 다르면 **inorder + preorder (또는 postorder)**로 유일하게 재구성할 수 있다
- → 다만 **BST는 key가 모두 다르면 preorder 하나 또는 postorder 하나만으로도** 유일하게 재구성할 수 있다 (BST property로 서브트리 key 범위를 정할 수 있기 때문)

Q3. Inorder + Preorder(전위 순회) 재구성 알고리즘, BST property에 의존?

> **힌트**

재구성은 Ch6에서 일반 binary tree에 대해 배웠다.

> **정답**

개념은 힌트 참고.

- **X**. 일반 binary tree에도 동작. BST property는 'inorder가 자동으로 정렬'이라는 추가 속성을 줄 뿐.

Q4. Sort in ascending order?

> **힌트**

BST의 존재 이유 — inorder 순회가 무엇을 출력하는가.

> **정답**

개념은 힌트 참고.

- **Answer**: Inorder traversal → $O(n)$
- → BST 자체가 '정렬된 자료구조' 역할

4. Insert

알고리즘

```
Insert(x, T):
    BST search 경로 따라 내려감
    NULL 도달하면 그 위치에 새 노드 생성
```

- **항상 leaf에 삽입** → 기존 구조 그대로, 맨 아래만 붙임

시간복잡도

- Search와 동일: **Balanced** $O(\log n)$ / **Skewed(편향)** $O(n)$

5. Delete — 3 Cases

핵심 아이디어

- 삭제할 노드의 **자식 수**에 따라 케이스 분리
- **목표**: BST property 유지하면서 제거

Case 1 — Leaf node (degree 0)

- 그냥 제거. 부모의 해당 child pointer를 NULL로.

Case 2 — Degree 1 node

- **유일한 자식이 그 자리를 대체** (부모와 직접 연결)

Case 3 — Degree 2 node (어려운 케이스)

> **방법**: 삭제 노드 z의 **in-order successor** s의 **(key, value) 쌍을 z에 함께 복사**한 뒤, s의 원래 노드를 삭제한다. predecessor를 쓰는 경우도 대칭이다.

- **In-order successor** = right subtree의 **최솟값** = right subtree에서 계속 왼쪽으로
- **In-order predecessor** = left subtree의 **최댓값** = left subtree에서 계속 오른쪽으로

중요: successor 노드는 반드시 degree ≤ 1

- **Why?**: '최솟값'이면 **left child가 없음** (있다면 그 left child가 더 작을 것)
- → successor는 최대 right child 1개만 → Case 1 or Case 2로 환원
- 따라서 **재귀적으로 또 degree 2를 만날 일 없음** → successor 삭제는 상수 작업

Q5. successor로 교체 후 BST property 저절로 유지?

> 힌트

successor는 오른쪽 서브트리의 최솟값 — 왼쪽 전체와 오른쪽 나머지 사이에 정확히 낀다.

> 정답

개념은 힌트 참고.

- **Yes**. Successor는 right subtree의 최솟값 → 자동으로 **left subtree 전부 < successor < right subtree 나머지** 성립

전체 Delete 복잡도

- Successor 탐색 = 높이만큼 내려감
- 전체: $O(h)$ (balanced $O(\log n)$, skewed $O(n)$)

6. SearchMinimum / Maximum

알고리즘

- **Min**: root에서 계속 **left**로 → leftmost node
- **Max**: root에서 계속 **right**로 → rightmost node
- 복잡도: $O(h)$

Q6. $O(1)$ Min이 가능한가?

> 힌트

Min은 leftmost 노드에 있다. 삽입·삭제 때마다 그 위치가 바뀐다면?

정답

개념은 힌트 참고.

- 최악 O(1) 보장은 일반적으로 **불가능**하다 — 일반 BST에서 Min은 O(h)이다.
- 단, 루트에 왼쪽 자식이 없으면 그 조회는 O(1)이다.
- 단, `minPointer`를 별도로 유지하면 O(1)이 가능하지만 Insert/Delete마다 갱신해야 한다.

7. Binary Tree → BST 변환

문제

- Tree **구조는 그대로** 두면서 **key 배치를 변경해** BST로 만들기 (key는 모두 서로 다르다고 가정). 각 노드의 (key, value)는 한 레코드로 함께 이동한다.

알고리즘

1. **Inorder traversal** → (key, value) 쌍 리스트 수집 O(n)
2. **리스트를 key 기준으로 정렬** ($O(n \log n)$)
3. **두 번째 inorder traversal** 중 정렬된 (key, value) 쌍을 순서대로 할당 O(n)

복잡도

- Inorder 2회 = $O(n)$
- 정렬 = $O(n \log n)$
- **Total** = $O(n \log n)$

왜 동작?

- **BST property ⟺ inorder = sorted sequence**
- 구조는 그대로 두고 inorder 순서가 sorted가 되게 (key, value) 쌍을 배치 → 자동으로 BST 성립
- 왜 동작? key가 모두 다를 때 **BST ⟺ inorder key 나열이 엄격한 오름차순**이기 때문
- **주의**: value 필드만 변경하는 것은 BST 변환이 아니다 — key(레코드)를 함께 이동해야 한다

8. BST 사용처 & 한계

BST가 적합한 경우

동적 집합에 대해 **Insert, Search, Delete, Range scan (sorted order 출력)** 모두 지원해야 할 때.

- **Priority Queue(heap)과 차이**:
  - Heap(힙): Min(또는 Max) 빠름. 부모-자식 대소만 보장해 단순 traversal로는 정렬 순서를 얻을 수 없지만, extractMin을 반복하면 O(n log n)에 오름차순으로 출력할 수 있다 (원래 Heap은 변경됨)
  - BST: 모든 key에 대한 search/update 가능, inorder 한 번으로 O(n)에 sorted output 가능
- **Hash table과 차이**:
  - Hash: $O(1)$ avg search/insert, **순서 없음**
  - BST: 순서 보존, range query 가능

한계: Unbalanced 문제

- 삽입 순서에 따라 skewed → worst $O(n)$
- **해결: self-balancing BST** (Ch10 AVL)

Q7. 중복 key 허용 시 문제?

> **힌트**

BST 규칙 "왼쪽 < root < 오른쪽"에서 등호가 들어가면 어떻게 되는가.

> **정답**

개념은 힌트 참고.

- 기본 property는 **left < node < right**로 중복을 허용하지 않아 규칙이 모호해진다
- 해결책 (convention):
  1. 방법 1 — 중복을 **right subtree**로 넣되, property를 **left < node ≤ right**로 변경해 일관되게 적용
  2. 방법 2 — Count 필드 추가
  3. 방법 3 — 허용하지 않거나 기존 레코드를 갱신

- → 어떤 rule이든 일관성이 중요

Summary

연산	Balanced	Skewed (worst)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Min/Max	$O(\log n)$	$O(n)$
Inorder (sorted)	$O(n)$	$O(n)$

- **핵심**: BST property + inorder = sorted
- **Delete 3 cases**: leaf / degree 1 / degree 2 (successor 교체)
- **문제**: 균형 유지 → 다음 챕터에서 AVL tree로 해결